DrawerSense: Single-Camera Robotic Clutter Clearing in Constrained Environments

1st Christopher Liem
Department of EECS
Massachusetts Institute of Technology
cliem@mit.edu

2nd Yibo Cheng
Department of EECS
Massachusetts Institute of Technology
yiboc@mit.edu

3rd Panagiotis Liampas
Department of EECS
Massachusetts Institute of Technology
pliam005@mit.edu

***Abstract*—This work presents DrawerSense, a single-camera robotic manipulation system designed to open a drawer, identify a target object within clutter, and retrieve it using unified perception and control pipeline. Unlike traditional multi-camera robotic setups, DrawerSense operates using only a gripper-mounted camera, requiring the system to reason under severe occlusion, restricted viewpoints, and highly constrained geometry. The proposed framework integrates three components: (1) drawer manipulation, where RANSAC-based plane estimation and antipodal grasp sampling enable the robot to localize and grasp the drawer handle, followed by compliant pulling for safe drawer extraction, (2) vision-based scene understanding, which employs SAM-CLIP and SAM3 segmentation models to generate object masks, reconstruct multi-view point clouds, and isolate the desired item in cluttered conditions, and (3) pick-and-place, where object-level reconstructions are used to evaluate antipodal candidates under kinematic and collision constraints. A finite-state machine coordinates the stages and handles failure conditions. Experimental results demonstrate that DrawerSense can successfully open the drawer, interpret the scene, and retrieve the desired object in a constrained environment using only a single-mounted camera.**

***Index Terms*—Clutter, VLM, SAM, RANSAC, Compliant motion, Robotic manipulation, Point cloud, Antipodal**

I. Introduction

Robotic manipulation systems traditionally rely on multiple fixed cameras positioned throughout the workspace to provide robust perception of the scene. These multi-camera setups offer wide coverage and strong geometric completeness, but they also introduce significant infrastructure requirements and reduce flexibility outside controlled laboratory environments. In contrast, many real-world settings - homes, offices, and warehouses - provide limited sensing capabilities, motivating the need for manipulation pipelines that operate effectively with minimal sensing.

An alternative approach is to equip the robot with a mounted camera. This offers greater mobility in the robot's sensing capabilities, allowing it to be deployed in generalized settings. However, this shift in infrastructure introduce significant challenges in perception ranging from self-occlusions, decreased sensing resolution, and difficult viewing positions. Developing manipulation pipelines that remain robust under such sensing constraints is an important topic within the industry setting.

In parallel, recent developments in vision models play a critical role in understanding the robot's environment. For example, it allows a robot to carry out complex tasks such as picking out and manipulating a desired object while leaving the others untouched. A successful vision model enables robots to detect and segment objects, inferring their spatial relationships which later influence planning and control. In conjunction with a mounted camera, robots are able to adapt to far more generalizable tasks by leveraging onboard perception to interpret diverse and unstructured environments.

Another interesting field of research is the problem of clutter-clearing, which is one of the fundamental challenges in robotic manipulation due to occlusions, partial visibility, and unpredictable object configurations. This problem becomes substantially more difficult when the clutter is contained within a structure such as a drawer, as the boundaries of the container restrict both the robot's motion and its available viewpoints, further limiting visual coverage and constraining feasible grasp strategies.

Thus, we aim to combine all the following topics into one project. Given a robot arm with a single mounted camera, we wish to (1) open the drawer without any external help (cameras), (2) identify an object of interest inside the drawer, and (3) retrieve the desired item while leaving the others untouched.

II. Related Work

A. Related Work on Clutter Clearing

Manipulating objects in cluttered scenes poses challenges related to occlusion, ambiguous geometry, and limited grasp visibility. Early grasp detection approaches leveraged RGB-D geometry to identify graspable surface patches and demonstrated reliable performance in dense clutter [1]. Deep learning approaches such as Dex-Net extended this line of research through synthetic dataset generation and analytic grasp quality models that improved robustness to uncertainty [2]. Subsequent work has explored the use of complementary non-prehensile actions - such as pushing or reorienting objects - to expose new grasp opportunities and minimize occlusion effect, enabling more reliable retrieval in densely packed environments [3]. While it may be interesting to explore the use of non-prehensile actions to manipulate the cluttered scene, this approach is out of scope for us especially given our camera constraints. Thus, we explore the clutter-clearing problem in

a simplified setting: given a drawer full of colored boxes, can we successfully extract the box with our desired color?

B. Related Work on Vision Models

The rapid development of vision foundation models (VLMs) has significantly improved open-set recognition and semantic understanding for robotic systems. The original Segment Anything Model (SAM) enables class-agnostic mask generation with strong generalization across image domains [4]. Extensions such as SAM-CLIP combine SAM's spatial understanding with CLIP's semantic understanding, improving segmentation reliability in cluttered scenes [5]. More recent models such as SAM3 doubles the accuracy of existing systems in image Promptable Concept Segmentation (PCS) and improves SAM's visual segmentation capabilities [6]. As such, we wish to evaluate the performance of SAM-CLIP and SAM3 in the context of robotic manipulation in clutter.

C. Related Work on Robot-Drawer Interaction

The problem of opening a drawer is particularly interesting because the vast number of possible approaches integrate unique robotic manipulation techniques. For example, recent work by *Gupta et al.* demonstrates an end-to-end system capable of opening previously unseen cabinets and drawers in real homes using a commodity mobile manipulator [7]. Their approach integrates visual perception, handle detection, and whole-body motion planning to execute the task under real-world variability. From a control-theoretic perspective, other researchers proposed an adaptive manipulation framework under significant uncertainty [8]. In particular, the avoid relying on prior knowledge of the kinematics of the drawer, such as whether the joint is prismatic or revolute, and instead estimates motion constraints online using force and torque feedback. This allows the robot to safely and effectively manipulate a wide range of articulated objects without predefined articulation parameters. Though it may have been interesting to experiment with a mobile manipulator, we are mainly experienced with the KUKA iiwa robot arm and thus opted to continue using it. In terms of the adaptive manipulation framework, we attempt to implement a similar idea in this paper and achieve a simplified model of their approach.

III. Approach

Our approach to the problem is divided into three stages. In the first stage, we explore the problem of opening a drawer using a single mounted camera. In the second stage, we understand the contents of the drawer and identify our desired object. In the final stage, we successfully manipulate our desired object while leaving the others untouched.

A. Drawer Manipulation

The drawer manipulation problem is split into three stages: (1) sampling a feasible grasp on the drawer handle, (2) maneuvering to the identified grasp, and (3) pulling the drawer open.

*1) **Sampling Feasible Grasps**:* Since our scene contains a single camera mounted on the robot gripper, the robot needs to move to a position where the camera can see the drawer. To achieve this, the user is able to specify a camera pose for the gripper in the World frame, ${}^{W}X^{G_{\text{camera}}}$, which is concatenated to the robot's initial pose ${}^{W}X^{G_{\text{initial}}}$ to produce a trajectory $[{}^{W}X^{G_{\text{initial}}}, {}^{W}X^{G_{\text{camera}}}]$. From there, a pseudo-inverse controller is used to maneuver the robot from the initial pose to the camera pose. It is important to note that the success of our further methods rely on whether the user specifies a reasonable camera-view pose for the robot to observe the scene.

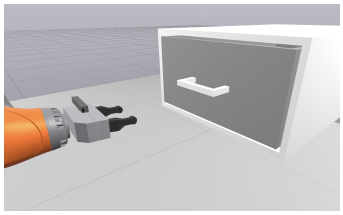


Fig. 1. The robot arm at the observation pose to record a point cloud of the scene.

After reaching the camera-view pose, the point cloud $\mathcal{P}$ is recorded. As of now, the table is also encoded in the point cloud which is undesirable. Thus, the raw point cloud $\mathcal{P}$ is filtered such that

$$\mathcal{P}' = \{p \in \mathcal{P} \mid p_z > z_{\text{min}}\}$$

where $z_{\text{min}} = 0.1$ in our setup.

Next, we wish to isolate the points that correspond to the drawer handle. In other words, we wish to further filter $\mathcal{P}'$ such that the drawer face is removed from the point cloud. To achieve this, Random Sample Consensus (RANSAC) is employed to fit a plane to the point cloud containing the drawer face and handle [9]. The algorithm is outlined in 1

To summarize, three random points in $\mathcal{P}'$ are sampled and used to produce a normal vector n. If $||n|| < \epsilon$, then that means that the three points are collinear and thus cannot uniquely define a plane. Otherwise, n is normalized and the distances from each point in $\mathcal{P}'$ to the plane are computed, keeping track of how many points are within a certain distance τ_{inlier} from the plane. The more inliers a plane has, the better the plane. This process is repeated for K iterations, and one of the best plane's sampled points is returned along with the plane's normal vector.

Since the drawer face is substantially larger than the handle, the plane that RANSAC returns fits the drawer face. Since the handle is extruded from the drawer plane, our point cloud $\mathcal{P}'$ can be further filtered as follows:

$$\mathcal{P}_H = \{p \in \mathcal{P}' \mid d(p) > \tau_{\text{handle}}\}$$

In particular, all the points that are within distance τ_{handle} of the plane are removed. τ_{handle} is a chosen parameters that should be less than the length of the handle's extrusion. This can be visualized in 2.

Algorithm 1 RANSAC Plane Fitting for Drawer Front Segmentation

Require: Filtered point cloud P', iterations K, inlier threshold τ_{inlier}, minimum inliers $n_{\min}$
Ensure: Plane parameters (p_0, n)

best_inliers $\leftarrow -1$
$p_0^\star \leftarrow$ None, $\quad n^\star \leftarrow$ None
for $i = 1$ to K **do**
  Randomly sample distinct points $a, b, c \in P'$
  $n \leftarrow (b - a) \times (c - a)$
  if $\|n\| < \varepsilon$ **then**
    continue ▷ degenerate triple
  end if
  $n \leftarrow n/\|n\|$
  Compute point–plane distances:

$$d(p) = |\, n^\top (p - a)\,|, \quad \forall p \in P'$$

  inliers $\leftarrow \#\{\, p \in P' \mid d(p) < \tau_{\text{inlier}} \,\}$
  if inliers $>$ best_inliers **then**
    best_inliers $\leftarrow$ inliers
    $p_0^\star \leftarrow a$
    $n^\star \leftarrow n$
  end if
end for
if best_inliers $< n_{\min}$ **then**
  warn: insufficient planar support
end if
return $(p_0^\star, n^\star)$



Fig. 2. The points p_i of $\mathcal{P}_H$ (in red) only lie on the drawer's handle.

To identify feasible grasps, the normals of each point are estimated using principal component analysis (PCA) [10]. In particular, for each point $p_i \in \mathcal{P}_H$, the k-nearest neighbors are collected to compute the sample covariance matrix:

$$\sum = \frac{1}{k} \sum_{j=1}^{k} (p_{i_j} - \overline{p}) \cdot (p_{i_j} - \overline{p})^\intercal$$

where k is the number of points considered in the neighborhood of p_i and $\overline{p}$ is the centroid of p_i's nearest neighbors. The surface normal, n_i is chosen as the eigenvector of $\sum$ corresponding to its smallest eigenvalue. Next, random pairs of points (p_1, p_2) are sampled with corresponding normals (n_1, n_2) to produce antipodal candidates for the grasp. In particular, if the sampled pair satisfies a distance constraint

$$d_{\min} \leq ||p_1 - p_2|| \leq d_{\max}$$

and also satisfies an antipodal constraint

$$n_1 \cdot n_2 \leq \tau_{\text{antipodal}}$$

then it is a valid antipodal grasp candidate. The distance constraint is meant to produce grasp points such that the robot can fit its gripper around the handle and the cosine similarity is used to constrain the feasible grasp points to opposite ends of the handle. For each candidate, a grasp is produced by setting the pose to be the average of the sampled points and the gripper is orientated to face the drawer plane. Candidate poses can be seen in 6.

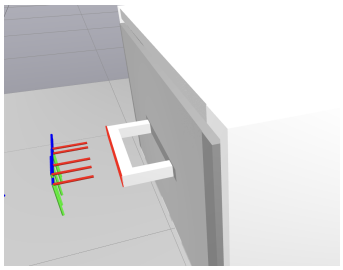


Fig. 3. Each coordinate frame corresponds to a unique candidate gripper pose.

*2) **Reaching Sampled Grasp**:* The next step is to maneuver the robot arm to reach one of the sampled grasp candidates. We don't have a heuristic for choosing one of the candidates, so one of the candidates is chosen at random since all of them are valid. This produces the gripper pose ${}^{W}X^{G_{\text{grasp}}}$, which is appended to the trajectory starting from ${}^{W}X^{G_{\text{camera}}}$. However, to ensure that the robot arm reaches the handle without collisions, a pre-grasp pose ${}^{W}X^{G_{\text{pre-grasp}}}$ is produced. This is simply attained by translating ${}^{W}X^{G_{\text{grasp}}}$ away from the drawer plane, which was found previously. Then, ${}^{W}X^{G_{\text{pre-grasp}}}$ is added to the trajectory such that we have waypoints $[{}^{W}X^{G_{\text{camera}}}, {}^{W}X^{G_{\text{pre-grasp}}}, {}^{W}X^{G_{\text{grasp}}}]$. This trajectory is differentiated to produce desired spatial velocities which are fed into the pseudo-inverse controller for manipulation.

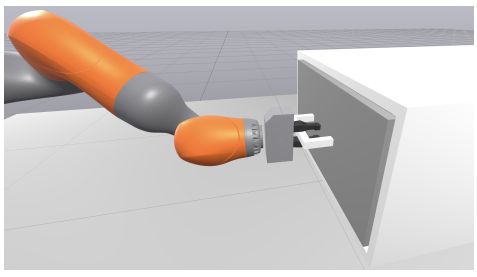


Fig. 4. The robot arm successfully gripping one of the candidate grasp poses shown in 6.

*3) **Pulling the Handle**:* Compliant motion is employed to open the drawer. The pipeline is quite simple: pull along the drawer-opening direction until the measured torques exceed a certain threshold. In particular, the desired spatial velocity ${}^{W}V^{G}$ is given by

$$ {}^{W}V^{G} = \begin{bmatrix} \omega \\ v \end{bmatrix},\ \omega = \mathbf{0} \in \mathbb{R}^3,\ v = v_{\text{nominal}} n^* \in \mathbb{R}^3 $$

where v_{nominal} is the desired translational pulling speed and n^* is the previously computed normal to the drawer plane. ω is set to the zero vector to prevent the robot arm from rotating while pulling the drawer. In terms of the stopping criterion, the system measures the joint torques $\tau_{\text{meas}} \in \mathbb{R}^7$ and computes their Euclidean norm

$$ ||\tau_{\text{meas}}|| = \sqrt{\sum_{i=1}^{7} \tau_i^2} $$

If the drawer is moving freely, then the resistance is low and the torque norm stays below a threshold τ_{stop}. However, if the drawer reaches its mechanical limits (meaning the drawer is fully opened), then the torque that the robot experiences rises due to friction. Thus, an adaptive controller switches between two modes:

$$ v_{\text{out}} = \begin{cases} v_{\text{nominal}} & \text{if } ||\tau_{\text{meas}}|| < \tau_{\text{stop}} \\ 0 \text{ otherwise} \end{cases} $$

The controller stops producing desired velocities once the drawer is fully open and the system transitions to the next state.

B. Vision

In order to get clear point clouds for each object in the scene, and be able to find a suitable grasp pose, we need to apply the following process:

- Get the segments of the image, from the gripper-mounted camera, that correspond to each object in the scene
- Get the point clouds corresponding to those segments, using the depth image, and get the points' world coordinates
- Concatenate the resulting point clouds over multiple views (images taken from various positions of the gripper), in order to get a more complete point cloud for each object, robust to occlusions from other objects of the scene. To that end, we are also downsampling the framewise (and the concatenated) point clouds, using a voxelized grid approach, since the number of points being processed would be too large otherwise.

Getting the world coordinates of each pixel of the image is quite a straightforward two-step process:

- Converting the depth value and pixel coordinates to the optical camera frame coordinates of that point, using the intrinsics matrix
- Converting the camera frame coordinates to world coordinates using the camera pose: ${}^{W}p^{O} = {}^{W}X^{C}\ {}^{C}p^{O}$

On the other hand, there is an important subtlety in this process that informed a lot of our design decisions in that aspect: Vision Transformers that are being used for image segmentation are not error-proof, especially when there are objects viewed from peculiar angles or heavily occluded. Thus, we need to find a way to verify which points correspond to which object over multiple views.

1) SAM-CLIP based approaches: There are different ways to achieve that goal; thus, we started from some simple approaches and worked our way to more complex ones. The first set of approaches that we tried was based on using the combination of a Segment Anything Model (SAM) that provides us with an extensive set of class-agnostic segments presented in an image, with a CLIP model that can compute embeddings for each of those segments. The importance of this principle is that we can aggregate the information provided by the CLIP embeddings over multiple views, allowing us to make a more well-informed decision for each point on whether it's part of any object in the scene.

A common factor in all these approaches is the computation of the cosine similarity between the CLIP embeddings of the cropped image segments and the CLIP embeddings of a pre-defined list of labels, which represent the objects that could be in the scene. This provides us with a metric on the likelihood/confidence that a segment corresponds to a certain type of object.

The simplest way we can combine that information is by applying a threshold on the computed cosine similarity, for each segment in every view, and concatenating (in the world frame) the respective point clouds of the selected segments for each label. This, in theory, would give us a perfect point cloud for each object. However, that's not the case, because, in the real world, the objects are occluded, and the masks are noisy; thus, we are also performing Euclidean distance-based clustering and picking the largest cluster for each label, in order to get a clear point cloud for each object of the scene.

Another approach, whose goal is to overcome the issue of potential misdetections of the segmentation model being used, is to first concatenate the point clouds resulting from each depth image, and then, using the stored segments and cosine similarities, compute the average cosine similarity for each point-label pair by projecting the point cloud back to the image corresponding to each segment and masking the points that are within it. Then, we assign each point to the label for which it has the highest average cosine similarity, as long as that similarity is above some threshold. Eventually, we switched to applying that threshold before computing the averages, because some segments that cover most of the image can significantly lower the averages, even for points that are parts of objects. Another subtlety that we had to handle was the fact that the projected points that were perceived to be part of a certain segment might have been part of the background instead, so we are also applying a threshold on the difference between the actual depth of that point and the respective depth of the pixel corresponding to it on the depth image.

Finally, for the third approach that uses the SAM and

CLIP combination, we are utilizing the concept that [11] introduced, that the cosine similarities can be interpreted as a probability that a point is part of a certain object with the given label, and that this information can be combined over multiple views using Bayesian Inference. We applied a variant of this approach for our project, however, we used a modified uniform prior (higher probability than the one used in Bayesian Fields) and normalized the probabilities of each point so that those for the different labels add up to 1. This lets us use the same principle with a smaller number of views (we were only working with 3 views).

2) SAM3 based approach: With the recent advanced on Vision Transformers, we have the ability to prompt a segmentation model, like SAM3, to provide the segments for each label separately, without the need to compute cosine similarities using CLIP embeddings, thus making our pipeline simpler. This approach essentially involves applying a similar procedure to the simpler SAM-CLIP based approach, but replacing the cosine similarity threshold with a confidence threshold on the scores produced by SAM3.

C. Pick and Place

Given the point cloud of the desired object returned by the vision pipeline, we evaluate a set of candidate antipodal grasps to identify a collision-free and kinematically feasible grasp pose.

First, the GenerateAntipodalGraspCandidate function is repeatedly used to sample grasp candidates. Each candidate returns both (i) a proposed gripper pose and (ii) a cost value reflecting grasp quality under antipodal heuristics. Samples that fail geometric criteria or return non-finite costs are immediately discarded.

Second, for all valid samples, we verify geometric feasibility by placing the gripper at the proposed pose inside the Drake plant context and performing signed-distance queries against the aggregated environment point cloud. If any point lies within a collision margin of the gripper, the candidate is rejected. This ensures that the antipodal configuration is not only theoretically valid but also physically executable without intersecting the surrounding geometry.

This procedure allows us to integrate perception-driven point-cloud data with physics-based feasibility checks, producing robust antipodal grasp proposals even under partial, noisy observations.

D. State Machine

To coordinate all three stages, we implement a robust finite-state machine composed of four principal stages: Handle Plane Estimation (RANSAC), Drawer Extraction via Compliant Control, Multi-View Object Reconstruction and Antipodal Grasp Selection, and Object Pick-and-Place Execution.

1) **Handle Plane Estimation:** the system applies RANSAC-based geometric fitting to detect the drawer handle. If the acquired point cloud lacks sufficient support for a reliable plane estimate or fails to yield a feasible antipodal grasp candidate, the state machine retains control in this stage and commands the robot to reposition the camera to obtain an alternative viewpoint before reattempting inference.
2) **Drawer Extraction via Compliant Control:** the robot performs a controlled pull to extend the drawer. Failure to achieve the prescribed displacement threshold triggers a full system reset.
3) **Multi-View Object Reconstruction and Antipodal Grasp Selection:** the manipulator executes a series of predefined scanning trajectories to accumulate denser point-cloud observations of the target object. If the resulting reconstructions remain too sparse to support stable antipodal grasp synthesis, the system iteratively samples new viewpoints; after five unsuccessful attempts, the state machine restarts to prevent deadlock.
4) **Object Pick-and-Place Execution:** When a high-quality grasp pose is obtained, this final stage is reached, where the planned grasp is enacted, and the object is transported to the top of the drawer.

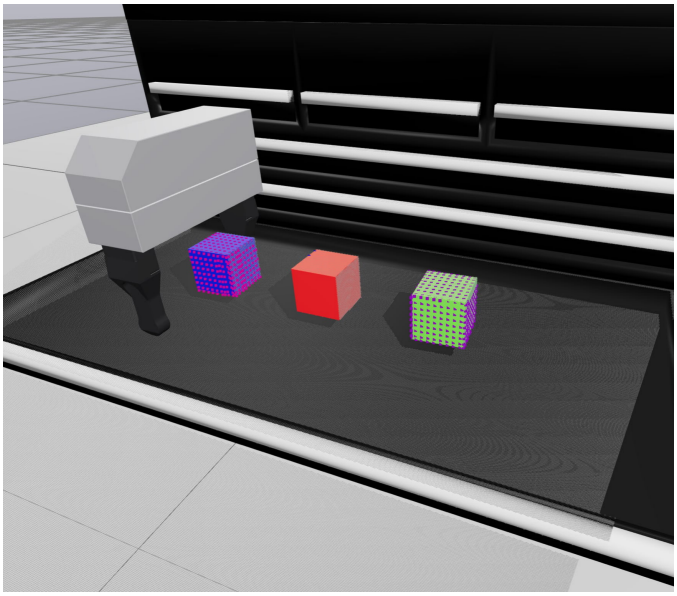


Fig. 5. Best antipodal grasp candidate for the purple box

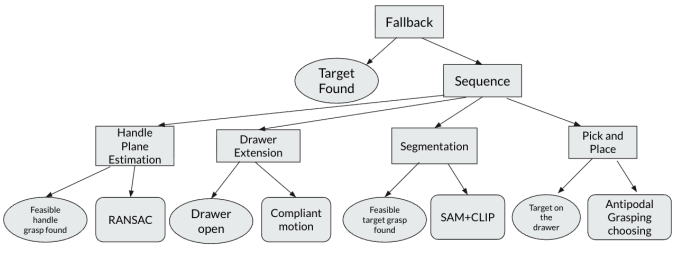


Fig. 6. State Machine

IV. Results and Discussion

A. Qualitative Results

Our manipulator successfully executes the full pipeline: it opens the drawer, segments the target object, and performs a reliable pick-and-place operation by transferring the object onto the top surface. Video demonstrations of the system in operation are available at this link

B. Perception Testing Procedure

Our testing consisted of both evaluating each aspect of our project separately, as well as integrating them together, and making sure our system works robustly.

Since a part of the project that also needs to be highly robust to different scene configurations (it has the highest uncertainty and is also dynamic) is the segmentation of the object point clouds, we made sure to test it on randomized configurations and evaluate the performance of the different approaches we considered before settling on using one of them. Specifically, we created a pipeline consisting of the following components:

- Generating random colored box configurations within the drawer
- Segmenting the resulting point cloud using all of the 4 Vision Transformer-based approaches, and evaluating the accuracy of such segments using the boxes' ground-truth positions (given by the simulator context)
- Trying to find valid grasp poses using the generated object point clouds, that don't have collisions with the rest of the environment
- Aggregating the success rates over all of the generated configurations

We then ran this pipeline for 20 randomly generated box configurations and gathered the results that are presented in the following section.

C. Quantitative Results

1) Evaluation of different clutter point cloud segmentation approaches: Overall, all 4 approaches achieved the desired goal of producing clear point clouds for the objects in the scene, as shown in Figure 5, even when they were occluded in some of the views taken by the gripper-mounted camera. However, when tested on 20 randomly generated scenes with 6 objects, we observe distinguishable results between those approaches, as seen in Figure 7.

SAM3 seems to perform better than the other approaches, resulting on a higher success rate in both accurately segmenting the point clouds of the objects in the scene, as well as finding a suitable grasping pose. This can probably be mostly attributed to the insufficiency of cosine similarities resulting from image and text CLIP embeddings to filter segments of objects that are not as apparent in the training dataset, like colored boxes, especially when viewed from simulated cameras, with a bit different color characteristics. This is especially apparent in the performance of the Bayesian Inference-based approach, where there is also the factor of an insufficient number of views being used, contrary to the implementation of [11].

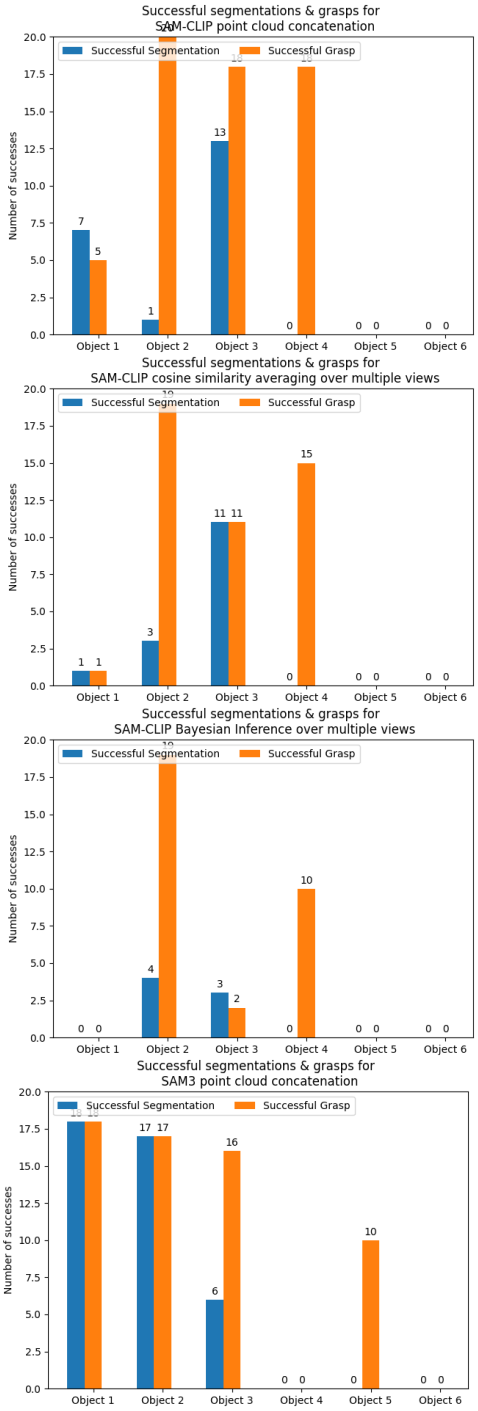


Fig. 7. Performance of the tested clutter point cloud segmentation approaches being run in 20 randomly generated environments. We can see that SAM3 performs better overall, both in terms of success in segmenting the point cloud of and grasping the respective object. Note that the grasping success rate is higher than the segmentation one in many cases, since the criterion on accurate segmentation excludes the cases where we are considering two objects to be the same, which still allows us to grasp at least one of them in most cases.

D. Limitations

Initially, we wished to use compliant motion to search for a feasible pose around the drawer handle. This beginning of the pipeline is similar to the methods previously discussed: maneuvering to an observation pose to get a point cloud of the scene and using RANSAC to identify the drawer plane. However, instead of isolating the drawer handle through direct point cloud processing, the robot arm would use the pseudo-inverse controller to touch the drawer using the computed plane. Next, we expected to be able to allow the robot to drag its gripper along the surface of the drawer while simultaneously checking for increased resistance. If the robot experienced an obstacle, then it would know that it found the handle since the handle is the only extruded feature of the drawer's face. Once the robot found the handle, then it would start rotating its gripper until it finds a robust grasp position. To pull the drawer open, the robot would employ the pulling process previously discussed.

When we tried to implement this pipeline, the robot arm often slid off the drawer plane since no constraints were imposed on the gripper's range of motion. As a result, the chances of finding the handle decreased significantly. Moreover, even if the handle was found, the robot would be unable to identify a robust grasp since we didn't have a well-defined heuristic that solely uses contact forces.

Thus, we transitioned to the simpler pipeline discussed in our approach. This process requires heavy parameter tuning and is contingent on the user feeding in an observation pose that was practically parallel to the drawer plane. If the point cloud was messy, then our threshold parameters would be more difficult to tweak. Since the point cloud is only used for identifying the drawer plane in our original algorithm, it would be more robust to the user's input observation pose. Thus, there are tradeoffs between strictly vision-based and strictly compliant-based approaches for grasping the handle.

Moreover, our methods rely on the assumption that the objects don't overlap each other, allowing SAM to easily segment the scene. If the objects were piled on top of each other, then additional searching algorithms would have been necessary to find our desired item. In particular, if our desired item was placed at the bottom of a pile of objects, then the robot arm would have to dig through all the clutter before seeing the objective in its line of sight. Methods such as those in [3] would have been necessary, but we were unfortunately unable to implement them due to time constraints.

To that end, there are trade-offs that might cause some segmentation approaches approaches to be more preferable than others in real-world conditions, and those depend on the number of cameras being used, the desired speed of the system (SAM3 seemed to be a bit slower than the SAM-CLIP combination), and the frequency and effect of occlusions between objects (which may hinder the ability for SAM3 to perform well in a single view, causing more misdetections).

V. Conclusion

In this work, we presented DrawerSense, a single-camera robotic manipulation system that autonomously opens a drawer, identifies a user-specified object within clutter, and retrieves it. By relying solely on a gripper-mounted camera, our system operates under severe sensing constraints and must compensate for self-occlusions, restricted viewpoints, and confined drawer geometry. We demonstrated how RANSAC plane estimation and antipodal grasp sampling enable robust handle localization and grasping, while a simple yet effective compliant pulling strategy allows the robot to open the drawer. On top of this, we integrated vision-based scene understanding using SAM-CLIP and SAM3, with antipodal grasp evaluation to performm reliable pick-and-place of the desired object.

Our experiments in a drawer cluttered with colored boxes shows that the full DrawerSense pipeline can successfully (1) open the drawer, (2) reconstruct object-level point clouds from a small number of views, and (3) execute collision-free antipodal grasps of the target object, placing it onto the drawer top. In particular, our comparison of multiple segmentation strategies indicates that SAM3 yields higher success rates in both point-cloud segmentation and grasping than SAM-CLIP-based approaches, though it comes with an increased computational cost. Moreover, our analysis highlights important limitations: the need for a reasonably chosen observation pose, the sensitivity of purely vision-based handle extraction to noisy point clouds, and the assumption that objects are not ehavily stacked or deeply occluding one another.

We hope that future works can improve our system using the methods discussed in the limitations section.

VI. Author Contributions

Christopher explored methods to open the drawer, Panagiotis experimented with vision models to understand the contents of the drawer, and Yibo implemented the manipulation of the objects in the drawer and integrated the system together.

References

[1] Gualtieri et al. High precision grasp pose detection in dense clutter (2016) https://arxiv.org/abs/1603.01564.

[2] Mahler et al. Dex-Net 2.0: Deep Learning to Plan Robust Grasps with Synthetic Point Clouds and Analytic Grasp Metrics (2017) https://arxiv.org/abs/1703.09312

[3] Zeng et al. Learning Synergies between Pushing and Grasping with Self-supervised Deep Reinforcement Learning (2018) https://arxiv.org/abs/1803.09956

[4] Kirillov et al. Segment Anything (2023) https://arxiv.org/abs/2304.02643

[5] Wang et al. SAM-CLIP: Merging Vision Foundation Models towards Semantic and Spatial Understanding (2023) https://arxiv.org/abs/2310.15308

[6] Carion et al. SAM 3: Segment Anything with Concepts (2025 https://arxiv.org/abs/2511.16719

[7] Gupta et al. Opening Cabinets and Drawers in the Real World using a Commodity Mobile Manipulator (2024) https://arxiv.org/html/2402.17767v1

[8] Karayiannidis et al. An Adaptive Control Approach for Opening Doors and Drawers Under Uncertainties (2016) https://ieeexplore.ieee.org/document/7398141

[9] Fischler et al. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography (1987) https://www.sciencedirect.com/science/chapter/edited-volume/abs/pii/B9780080515816500702

[10] Estimating Surface Normals in a PointCloud https://pcl.readthedocs.io/projects/tutorials/en/pcl-1.12.0/normal_estimation.html#:~:text=The%20solution%20for%20estimating%20the,the%20covariance%20matrix%2C%20and%20the
[11] Maggio, D., & Carlone, L. (2025). Bayesian fields: Task-driven open-set semantic gaussian splatting https://arxiv.org/pdf/2503.05949